

16+



XXVIII Всероссийская
(с международным участием)
молодежная научно-практическая
конференция
Низневартовского
государственного университета



г. Нижневартовск
2026

г. Нижневартовск, 8–9 апреля 2026 г.

Часть 2. Техника и технологии

РАЗРАБОТКА ВЕБ-САЙТА ДЛЯ АЭРОПОРТА ГОРОДА НИЖНЕВАРТОВСКА

В условиях активной цифровизации транспортной инфраструктуры наличие информативного, удобного и технологичного веб-сайта является ключевым инструментом взаимодействия аэропорта с пассажирами. Сайт позволяет предоставлять актуальную информацию о рейсах, услугах, правилах и контактах, а также обеспечивает основу для внедрения дополнительных онлайн-сервисов. В данной статье представлена разработка веб-сайта для аэропорта города Нижневартовска, основанная на современном стеке технологий с применением контейнеризации [3; 5] для обеспечения стабильности, масштабируемости и удобства развертывания. Для реализации проекта был выбран следующий набор технологий:

- Frontend: HTML5, CSS3, JavaScript – для построения интерфейса, обеспечения адаптивности и интерактивности.
- Backend: Node.js с фреймворком Express – для создания серверной логики и API [1].
- Шаблонизация: EJS (Embedded JavaScript) – для динамической генерации HTML на стороне сервера с возможностью повторного использования компонентов.
- Контейнеризация: Docker – для упаковки приложения и зависимостей в изолированные контейнеры, что упрощает развертывание и обеспечивает единообразие среды выполнения [3].

Приложение состоит из нескольких сервисов, каждый из которых работает в отдельном Docker-контейнере. Вспомогательные микросервисы (Node.js + Express) – Обрабатывают запросы по дополнительным данным, таким как суточное расписание полетов, сезонное расписание и т. д. Такое разделение на микросервисы позволяет масштабировать каждый компонент независимо и повышает отказоустойчивость системы [4] (рис. 1). Архитектурный подход является гибридным [2]:

- SSR (Server-Side Rendering) реализован с использованием EJS в качестве основного шаблонизатора, который выступает в роли мастер-страницы (masterPage), обеспечивая единообразное отображение шапки (header) и подвала (footer) на всех страницах сайта.
- CSR (Client-Side Rendering) применяется для динамических компонентов, таких как онлайн-табло или блок услуг.

Данные для этих компонентов хранятся на сервере в формате JSON и могут обновляться администратором без изменения клиентского кода, что упрощает поддержку и редактирование контента. Главная цель при проектировании была создать максимально простой инструмент, с минимальными зависимостями (от библиотек, фреймворков и т. д.) и легкой масштабируемостью. Поскольку проект на данном этапе лишь черновой вариант, в дальнейшем вполне возможны изменения в архитектурных подходах, дизайнерских решениях и так далее. Далее будут представлены схемы взаимодействия.

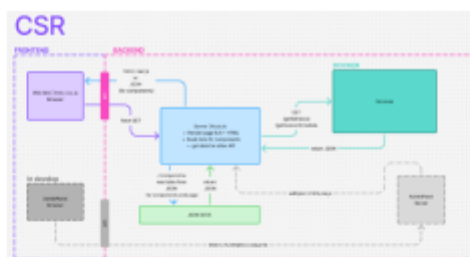


Рис. 1. Архитектура проекта сайта аэропорта



Рис. 2. Пример логики взаимодействия client/server

В веб-приложении реализована гибкая система обработки клиентских запросов и динамического рендеринга компонентов. При обращении клиента на сервер настроено несколько маршрутов (routes), через которые клиент может получать данные о страницах, данные для компонентов и осуществлять взаимодействие со сторонними сервисами (рис. 2). Рассмотрим механизм обработки запросов страниц на примере кода сервера. Когда клиент отправляет запрос на адрес <http://nv-aero.ru/home> (например, запрашивая домашнюю страницу), сервер выполняет следующую последовательность действий:

- Получение параметра запроса: сервер принимает в качестве параметра идентификатор (id) страницы, которую необходимо найти и отобразить.
- Загрузка структуры страниц: сервер считывает всю структуру страниц сайта, которая хранится в JSON формате (рис. 4). Представляет собой плоскую структуру, описание всех страниц сайта с их свойствами: идентификаторами, названиями, метаданными, списками компонентов и статусом видимости.
- Кэширование структуры (модуль в разработке): для оптимизации производительности реализован механизм кэширования структуры страниц. Это позволяет избежать повторного чтения и парсинга файлов структуры при каждом запросе.

- Фильтрация страниц: сервер фильтрует загруженную структуру по заданному идентификатору и дополнительному условию – страница должна быть активной (видимой для пользователей).
- Возврат структуры для рендеринга: найденная структура страницы передается в шаблонизатор EJS для генерации HTML-кода. EJS использует мастер-шаблон (masterPage), который обеспечивает единообразное оформление всех страниц сайта (рис. 3).

[illegible]

Рис. 3. Листинг кода на сервере для отрисовки страницы

[illegible]

Рис. 4. Структура хранения страниц для сайта

После того как сервер отдал HTML-страницу, начинается процесс динамического рендеринга компонентов на стороне клиента. На каждой странице присутствуют собственные компоненты. В случае со страницей с id:home такими компонентами являются:

- Новости – блок актуальных новостей аэропорта
- Сервисы – список доступных услуг
- Табло – динамическое расписание рейсов

Выбранный подход – разделение рендеринга страницы на серверную и клиентскую части – имеет несколько важных преимуществ [2]:

1. Упрощение серверной структуры: сервер отвечает только за базовую структуру страницы, что делает его код более чистым и понятным.
2. Читаемый формат кода: компоненты реализованы на чистом JavaScript и HTML, без использования сложных шаблонизаторов типа EJS или виртуального DOM. Это облегчает

понимание и поддержку кода, хотя и приводит к дополнительной нагрузке на клиенте из-за необходимости рендеринга на стороне браузера.

3. Гибкость обновления: компоненты могут независимо обновлять свои данные без перезагрузки всей страницы.

Рассмотрим подробнее логику компонента на примере компонента «Сервисы» – услуги аэропорта:

1. Инициализация места для рендеринга: в HTML-коде страницы оставляется специальный контейнер для отображения компонента, мы просто даем уникальный id любому тегу html, где захотим рендерить компонент (рис. 5).

```
<section class="h-screen px-20 py-10 bg-[#cdd5dd]" id="serversNav">
  <h2 class="text-xl md:text-4xl font-bold text-center mb-12 text-[#014e6c]">Сервисы </h2>
  <div id="servers" class="grid md:grid-cols-2 grid-cols-1 h-1/2 gap-6"></div>
</section>
```

Рис. 5. Инициализация контейнера для сервисов аэропорта

2. Запрос данных компонента: после загрузки страницы каждый компонент самостоятельно запрашивает данные для отрисовки с сервера через специальные API-маршруты. Например, для компонента «Сервисы» запрос отправляется по адресу <http://nv-aero.ru/pages/componentsInfo/servers> (рис. 6).

```
1 import { useState, useEffect } from 'react';
2
3 import axios from 'axios';
4
5 const API_URL = 'http://nv-aero.ru/pages/componentsInfo/servers';
6
7 export default function Servers() {
8   const [servers, setServers] = useState([]);
9   const [loading, setLoading] = useState(true);
10
11   useEffect(() => {
12     fetchServers();
13   }, []);
14
15   const fetchServers = async () => {
16     try {
17       const response = await axios.get(API_URL);
18       setServers(response.data);
19     } catch (error) {
20       console.error('Error fetching servers:', error);
21     }
22   };
23
24   return (
25     <div>
26       <h2>Сервисы</h2>
27       <div>
28         {servers.map((server) => (
29           <div key={server.id}>
30             <h3>{server.name}</h3>
31             <p>{server.description}</p>
32             <p>{server.location}</p>
33             <p>{server.schedule}</p>
34             <p>{server.price}</p>
35           </div>
36         ))}
37       </div>
38     </div>
39   );
40 }
```

Рис. 6. Листинг кода компонента servers на странице home

3. Рендеринг компонента на клиенте: Получив данные от сервера, клиентский JavaScript:

- Парсит JSON-ответ.
- Генерирует HTML-структуру на основе полученных данных (рис. 7).
- Вставляет сгенерированный HTML в заранее подготовленный контейнер.
- Настраивает интерактивные элементы (кнопки, фильтры, сортировку).

[illegible]

Рис. 7. Листинг кода светлого дизайна компонента servers на странице home

Такой архитектурный подход, сочетающий серверный рендеринг базовой структуры страницы и клиентский рендеринг динамических компонентов, обеспечивает оптимальный баланс между скоростью загрузки, интерактивностью и удобством поддержки кода [4].

Далее рассмотрим дизайн. Основные модули сайта. Главная страница: Информационный дашборд с виджетом онлайн-табло вылетов/прилетов и их статусом на данный момент. Динамическое расписание рейсов (рис. 8).

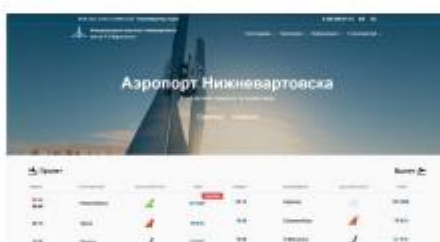


Рис. 8. Стартовая страница сайта аэропорта города Нижневартовска

Выпадающий список: для каждого раздела соответствующий набор информационных компонентов (рис. 9).

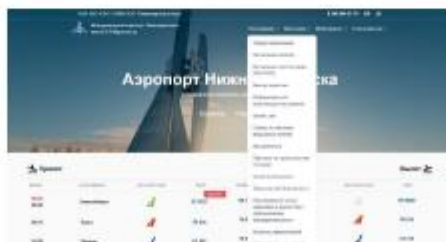


Рис. 9. Список разделов сайта аэропорта

Услуги аэропорта: Структурированный раздел с описанием услуг (парковка, VIP-залы, магазины), включая возможность онлайн-бронирования (рис. 10).

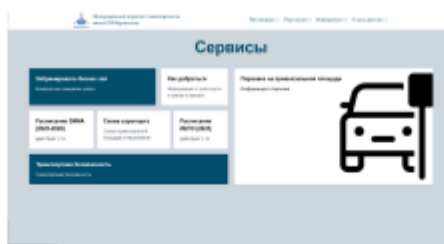


Рис. 10. Блок сервисов (услуг) сайта аэропорта города Нижневартовска

Новости: Блок актуальных новостей аэропорта (рис. 11).

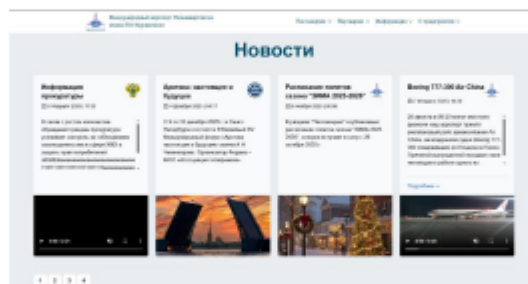


Рис. 11. Блок новостей сайта аэропорта

В заключение, разработанный прототип веб-сайт для аэропорта Нижневартовска представляет собой современное, производительное и масштабируемое решение, которое полностью соответствует потребностям пассажиров и сотрудников. Использование гибридного архитектурного подхода (SSR/CSR) в сочетании со стеком HTML/CSS/JS, EJS, Node.js и Docker [1-3] позволяет обеспечить высокую скорость разработки, удобство поддержки и отказоустойчивость системы. Гибкая микросервисная архитектура обеспечивает возможность дальнейшего расширения функциональности без переписывания существующего кода [5].

Литература

1. Иванов А.В. Современные подходы к разработке веб-приложений на Node.js // Вестник информационных технологий. 2023. № 4. С. 45-52.
2. Коваленко П.И. Гибридные подходы к рендерингу в современных веб-приложениях // Информационные системы и технологии. 2024. № 1(45). С. 23-31.



3. Петрова Е.С. Контейнеризация приложений с использованием Docker: практический опыт // Труды международной конференции «Информационные системы 2024». М.: Изд-во МГУ, 2024. С. 112-118.
4. Смирнов Д.К. Адаптивный дизайн и юзабилити транспортных веб-ресурсов // Интернет-маркетинг. 2022. № 6. С. 34-40.
5. Федоров М.И. Микросервисная архитектура в веб-разработке // Программная инженерия. 2023. Т. 14, № 2. С. 78-85.

© Валентюкевич О.Е., 2026